

PROJECT OVERVIEW

CelerisDB

A distributed database that lets each request choose between "always correct" and "always available".

This guide explains, in plain language, what CelerisDB is, how its pieces fit together, how a request travels through it, and what makes it different.

What is CelerisDB?

CelerisDB stores **JSON documents under keys**, like `orders/42 → {"status":"paid","total":120}`. It runs as one small program on one laptop or on many servers at once. When it runs on many servers (a **cluster**), it keeps several copies of every piece of data, so the data survives a crashed machine, a broken disk or a cut network cable.

Every distributed database must answer one hard question: *when the network breaks, do you stop accepting writes (to stay perfectly correct), or keep accepting them (to stay available) and sort things out later?* Most databases make that choice once, for everybody. **CelerisDB lets every single request choose**, and it is honest about what happened when it cannot be sure.

5

consistency modes, chosen per request

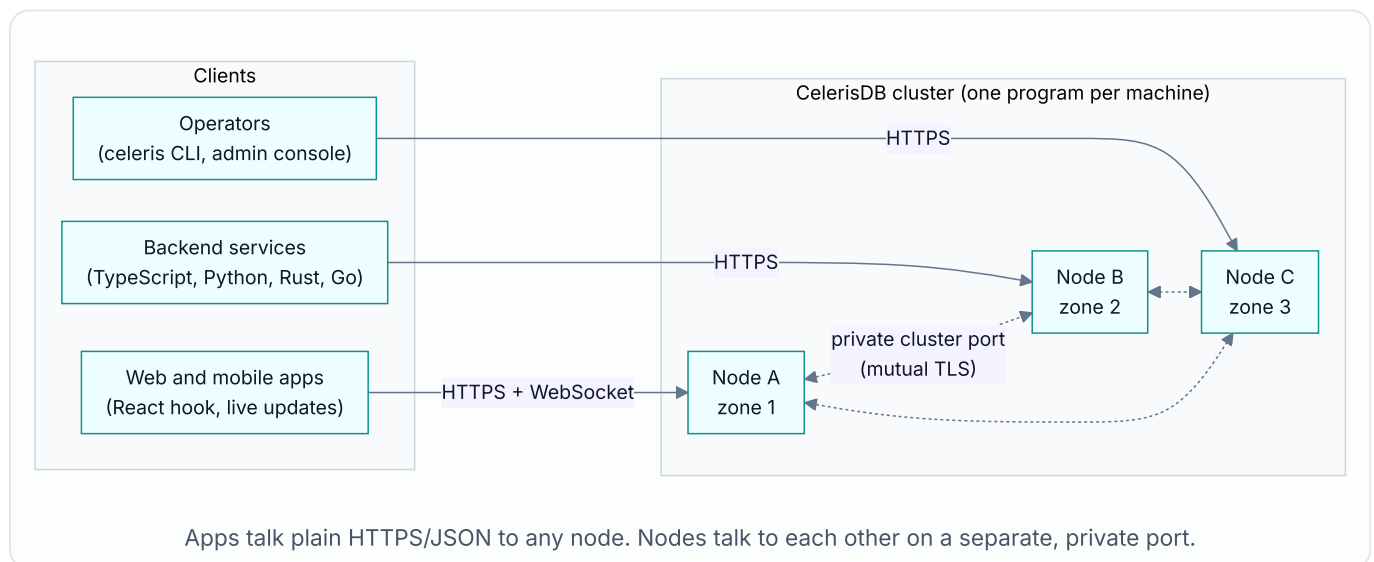
4,096

partitions spread across machines

4 SDKs

TypeScript (+React), Python, Rust, Go

The idea in one picture

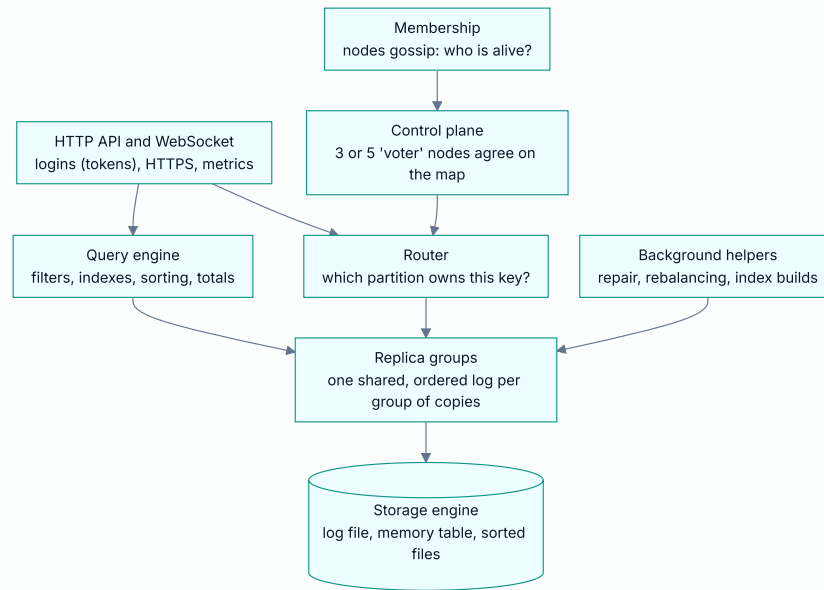


Who is it for?

- **App teams** who want a simple key/JSON store with live updates and safe retries out of the box.
- **Platform teams** who need data to stay correct through failures, and to keep working in a degraded network when the business prefers that.
- **Self-hosters**: one binary, runs on a laptop, Docker, Kubernetes or AWS; the data stays on your machines.

Architecture: what is inside one node

Every machine runs the same program, called a **node**. A node is made of a few layers, each with one job:



The layers inside a node. Arrows show who asks whom.

1. API layer

Speaks HTTP and JSON, so browsers, scripts and SDKs can all use it. It checks API tokens, serves HTTPS, and streams live changes over WebSockets.

2. Router

Every key is hashed into one of **4,096 partitions**. A shared **partition map** says which machines hold each partition, so any node can send a request to the right place.

3. Replica groups

Each partition has 3 copies on 3 machines (a **replica set**). The copies agree on the order of writes with the **Raft** algorithm: one copy leads, a majority must confirm each write.

4. Storage engine

Written from scratch in Rust. It saves each write to a log file on disk first (so nothing acknowledged is lost), keeps recent data in memory, and moves it into sorted files.

5. Control plane

3 or 5 "voter" nodes run their own Raft log that holds the partition map. Changing who owns what is a decision a majority must agree on, so nodes never disagree about ownership.

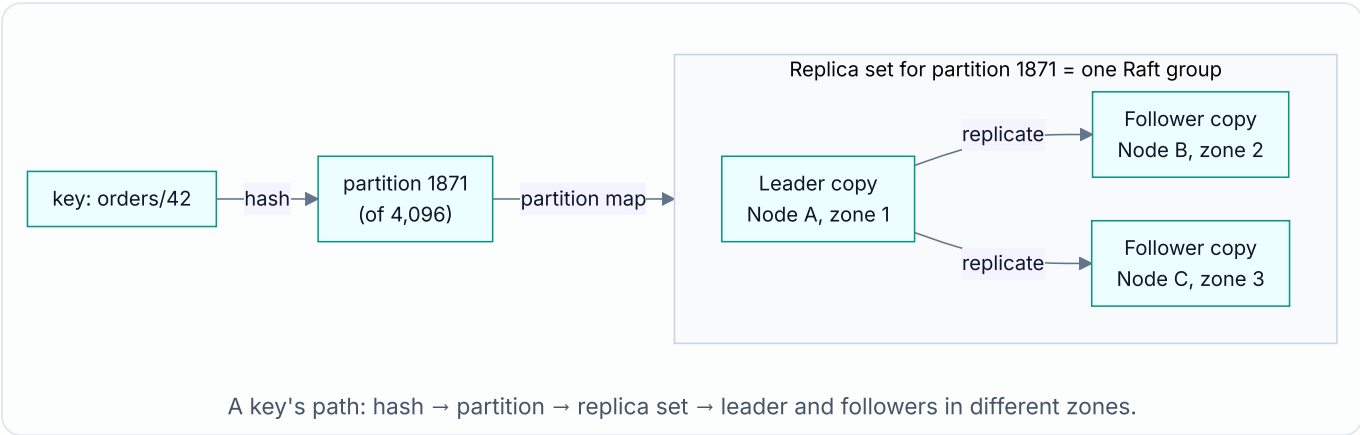
6. Membership

Nodes constantly ping each other (**SWIM gossip**). A silent node is first "suspect", then "unreachable", and its partitions are moved elsewhere automatically.

Novelty — a Raft group per replica set. Instead of one giant log for the whole cluster (a bottleneck) or none at all (no strong guarantees), CelerisDB runs one small Raft log per group of copies, plus one for the map. Groups work in parallel, and each stays strictly ordered.

Runtime topology: how data is spread out

Here is what a typical 3-machine cluster looks like while running. Each machine sits in a different **zone** (a rack, or an AWS availability zone), so losing one zone never loses data.



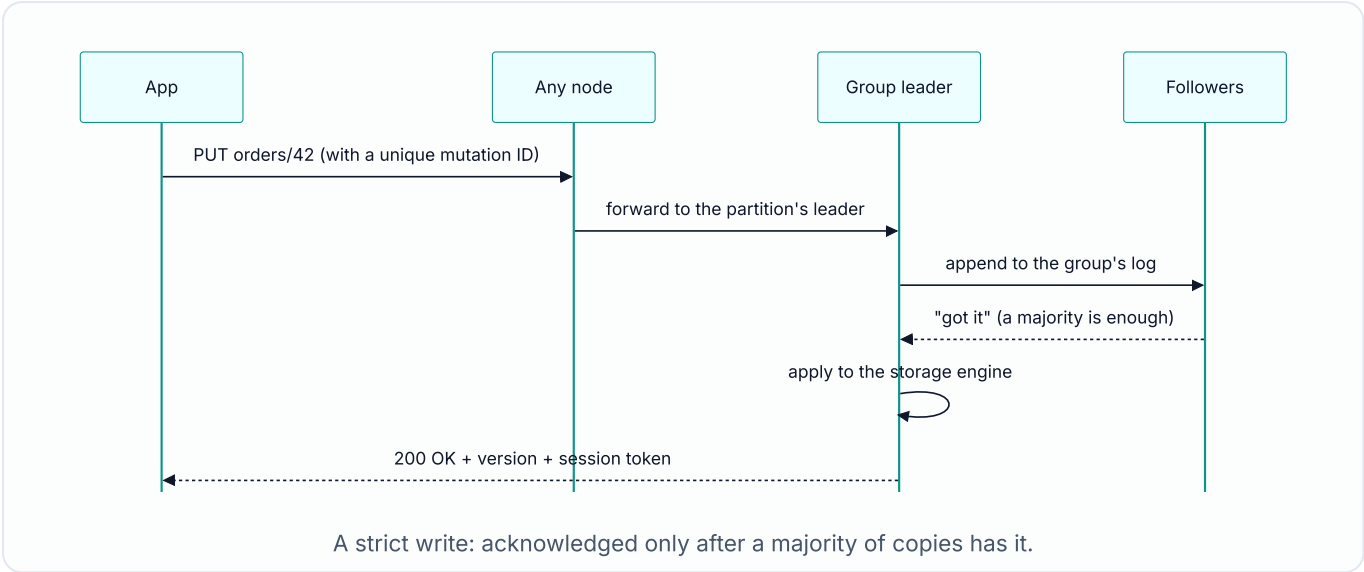
How partitions are placed

- **Placement is calculated, not stored per key.** A formula ("rendezvous hashing") picks machines for each partition, spreads the copies across zones, and keeps load even (within about 10–15%).
- **Adding a machine moves only about 1/N of the data,** and only to the new machine. Removing one moves only its own partitions.
- **Epoch numbers fence old owners.** Every map version has a number. A node holding an older map is told so and retries, so two nodes never both think they own a partition.
- **Moves happen while serving traffic.** A partition being moved is briefly frozen, copied in chunks, then switched over. Clients just retry the few writes that hit the switch.
- **Self-healing.** If a machine stays unreachable, the control plane re-replaces its partitions after a delay (so a short blip does not cause a storm of moves).
- **Anti-entropy.** Leaders regularly compare fingerprints (hashes) of every copy. A copy that drifted is repaired from a fresh snapshot.

The ports a node uses

Port	What travels on it	Who should reach it
8080	The HTTP/JSON API, live WebSocket streams, <code>/metrics</code> , <code>/health</code> , <code>/ready</code>	Apps, load balancers, Prometheus
7000	Gossip, Raft messages, snapshots, partition moves (optionally with mutual TLS)	Other nodes only

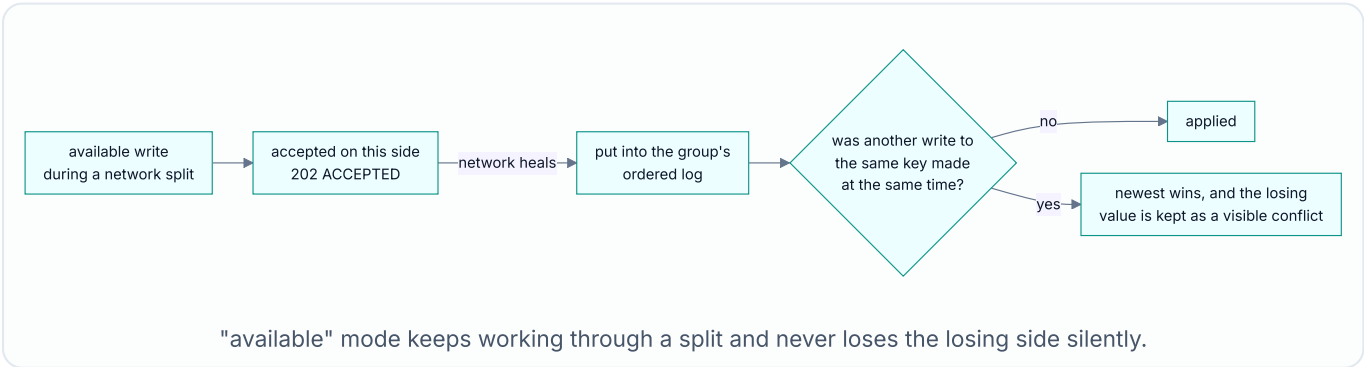
How it actually works: a write, step by step



- **Safe retries.** Every write carries a **mutation ID**. If the network drops the answer and the app retries, the database recognizes the ID and returns the original result instead of applying the write twice.
- **Honest failures.** If a node truly cannot know whether a write landed (for example, it crashed mid-way), it says `outcome_unknown` instead of guessing. The app can ask later: "did mutation X commit?"
- **Compare-and-set.** "Only write if the version is still 7" makes safe counters, carts and bookings possible without locks.

Choose your consistency, per request

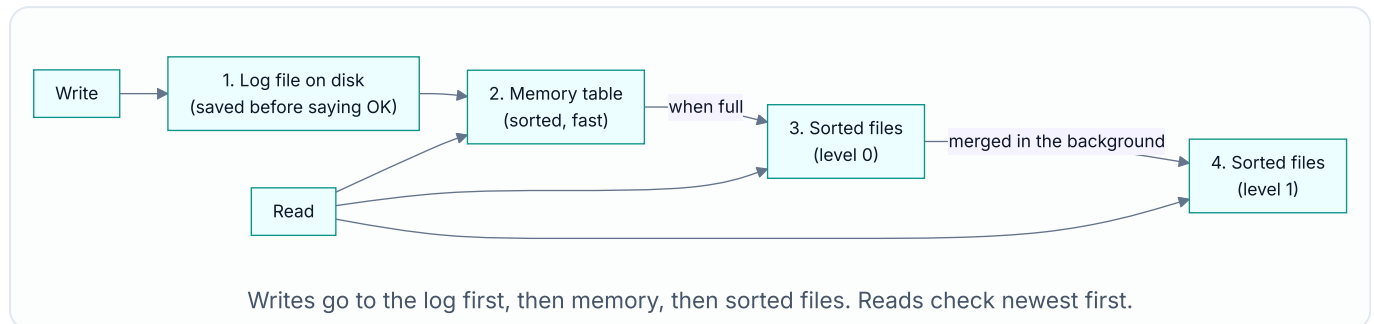
Mode	In plain words	Good for
strict	Always the latest value; the leader double-checks it is still the leader before answering.	Payments, inventory
session	You always see your own writes, even on a different node (the client carries a small token).	User profiles, settings
bounded	Possibly stale, but never older than a limit you choose, e.g. 500 ms.	Dashboards
available	Writes are accepted even when the cluster is split; they are merged once it heals.	Likes, carts, telemetry
eventual	Fastest read from any copy; it catches up shortly.	Feeds, caches



Novelty — availability without silent data loss. Many systems resolve concurrent writes with "last writer wins" and simply drop the other value. CelerisDB records every losing write as a **conflict** you can list (`/v1/conflicts`) and clear, so nothing disappears without a trace.

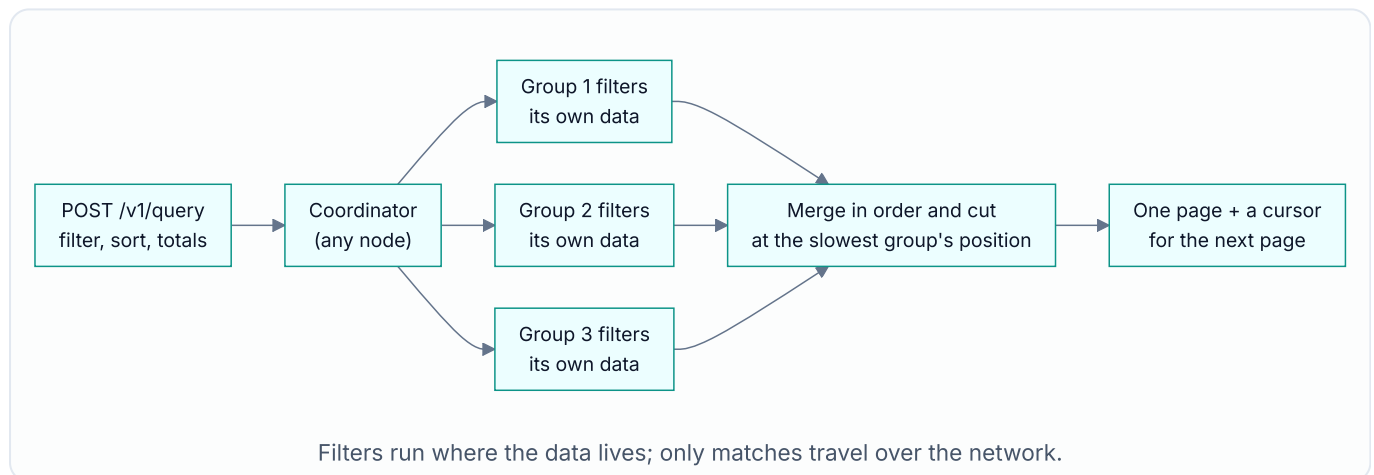
The storage engine, in plain words

Each node stores its data with its own engine, built from scratch (a "log-structured merge tree", the same family as RocksDB and Cassandra's storage):



- **Crash-safe:** after a crash, the log is replayed. Every file has a checksum and a format version; damaged data is detected, never silently used.
- **Fast lookups:** each sorted file has a **Bloom filter** (answers "definitely not here" in about 0.26 μ s) and a cache of recently used blocks.
- **Group commit:** forcing data to disk is the slow part. When many writers arrive together, one disk flush covers all of them: 3–5 $\times$ more safe writes per second.
- **Expiry (TTL):** keys can expire automatically, and deleted data is cleaned up during merges.
- **Backups:** a consistent online snapshot of a node (`celeris backup`) without stopping writes, plus JSON-lines export/import for whole clusters. Re-running an import is safe.

Finding data: queries, indexes, sorting and totals



- **Filters** in a familiar MongoDB-like style: `{"status":"paid","total":{"$gte":100}}` , with and/or/not, nested fields and projections.
- **Secondary indexes** on a JSON field make "status = paid" read only the paid orders. **Sorting** (ascending or descending) walks an index in order.
- **Totals:** count, sum, minimum and maximum, merged across pages.
- **Bounded work:** every request reads at most a set number of rows and returns a cursor, so one big query cannot overload a node.

Novelty — indexes kept exact inside the same disk write. The storage engine updates index entries in the *same atomic commit* as the data, so every path (API writes, replication, partition moves, repairs) keeps indexes correct automatically. Building a new index runs as small replicated steps, so all copies stay identical.

Tech stack

Part	Built with	Why
Database core	Rust (async runtime: Tokio)	Speed close to C, with memory safety; no garbage-collector pauses
HTTP API	axum + hyper; WebSockets	Mature, fast HTTP stack; live change streams
Security	rustls (TLS), SHA-256 hashed tokens	Modern TLS without OpenSSL; secrets never stored in plain text
Consensus and membership	Raft and SWIM, written in-house	Full control over failover, pre-vote and leader stickiness
Storage	Custom LSM engine (WAL, memtable, SSTables, Bloom filters, xxHash, CRC32)	Built for this workload, tested against crashes
SDKs	TypeScript (+ React hook), Python, Rust, Go	Same safety rules (retries, sessions, honest outcomes) in every language
Admin console and website	React + Vite; GSAP and Lenis for the website's animation	Fast, modern web UIs
Operations	Docker, Docker Compose, Kubernetes, GitHub Actions, Prometheus metrics	Runs the same way on a laptop and in the cloud

How we know it works

Linearizability checker

A test records thousands of real operations during leader crashes, then mathematically checks that the results could have happened in one correct order.

Chaos suite

Nodes are cut off from the network, healed and crashed at random while clients update counters. Afterwards, every counter must match exactly what was acknowledged.

Crash recovery

A real process is killed mid-write, over and over. On restart, every acknowledged write must be there, and nothing torn.

Benchmarks

Micro-benchmarks track the engine: about 156K writes/s in memory, about 0.26 μ s to rule out a missing key, about 4 million rows/s scanned.

Novelty — chaos testing inside one process. Network splits are simulated by each node refusing to talk to chosen peers, so a realistic failure test runs in seconds on a laptop or in CI, with a seed to replay any failure exactly.

Novelties at a glance

1. **Consistency chosen per request**, from strict to eventual, including writes that keep working during a network split.
2. **No silent data loss**: concurrent losing writes are kept as visible conflicts.
3. **Honest outcomes**: "unknown" is reported as unknown, and mutation IDs make every retry safe.
4. **One Raft group per replica set** plus a Raft-run partition map, with zone-aware placement.
5. **Engine-level indexes** updated in the same atomic commit as the data, built through replicated steps.
6. **Gap-free paging across machines** for filtered and sorted queries, with bounded work per request.
7. **Descending sort without reverse scans**, by storing an inverted copy of the sort key.

All features of CelerisDB

Data and API

- JSON values under string keys; put, get, delete
- Atomic batches of many writes
- Compare-and-set (if version / if absent)
- Time-to-live (automatic expiry)
- Ordered scans by prefix or key range
- Mutation IDs: exactly-once retries, status lookup

Consistency and availability

- strict, session, bounded, available, eventual
- Writes during network splits, merged later
- Recorded, listable, clearable conflicts
- Automatic repair of drifted copies
- Automatic leader election and failover

Cluster

- 4,096 partitions, zone-aware placement
- Raft per replica set, Raft control plane
- Gossip membership and failure detection
- Live partition moves and automatic rebalancing
- Automatic first placement when the cluster forms

Queries

- MongoDB-style filters with nested fields
- Field projection
- Secondary indexes (built in the background)
- Sorting, ascending or descending
- Count, sum, min, max
- Live change streams (WebSocket) by key prefix

Security and operations

- API tokens with read / write / admin scopes
- HTTPS for clients, mutual TLS between nodes
- Online backup and restore; export / import
- Prometheus metrics, health and readiness checks
- `celeris` CLI: init, start, doctor, bench, query and more
- Web admin console

Developer platform and deployment

- SDKs: TypeScript + React hook, Python, Rust, Go
- Session tokens and safe retries built into SDKs
- Docker image and 3-node Docker Compose cluster
- Kubernetes StatefulSet with disruption budget
- AWS reference architecture (EC2 / EKS)
- Release binaries and one-line installers

Getting started in one minute

`celeris init` → `celeris start` → `celeris put users/42 '{"name":"Ada"}'` → `celeris get users/42` .
For a cluster: `docker compose up -d` starts three nodes that find each other and place partitions on their own.

Known limits today. Sorting needs an index on the sorted field. Very large integers (beyond 2^{53}) can tie in sort order. Certificates are reloaded only on restart.